

Практическое занятие 5. Общие табличные выражения в PostgreSQL

1 Цель занятия:

Освоить технологию общих табличных выражений Common Table Expressions (CTE) в PostgreSQL как мощный инструмент структурирования сложных SQL-запросов. Научиться применять обычные CTE для улучшения читаемости и поддерживаемости кода, а также рекурсивные CTE для обработки иерархических данных и сложных аналитических расчетов. Сформировать навыки использования CTE в операциях модификации данных и оптимизации запросов, что в совокупности позволит эффективно решать широкий спектр практических задач обработки информации в реляционных базах данных.

2 Основные задачи:

- Изучить синтаксис и принципы работы Common Table Expressions, включая создание обычных и рекурсивных CTE, их область видимости и время жизни.
- Освоить практическое применение CTE для структурирования сложных запросов, улучшения читаемости кода и организации многоэтапных вычислений в аналитических задачах.
- Научиться разрабатывать рекурсивные запросы для обработки иерархических данных, древовидных структур и последовательностей с контролем глубины рекурсии.
- Применить CTE в операциях модификации данных (INSERT, UPDATE, DELETE) для реализации сложной бизнес-логики с гарантией целостности и трассируемости изменений.
- Проанализировать реальные кейсы использования CTE в заданной предметной области «Сессия» для решения задач анализа успеваемости, планирования нагрузки и управления академическими данными.

3 Теоретическая часть

3.1 Общее табличное выражение

Общее табличное выражение – Common Table Expression (CTE) – это временный результат выполнения SQL-выражения, который можно использовать в другом SQL-выражении. CTE позволяет упрощать сложные SQL-запросы, разбивая их на составные части. CTE определяется с помощью выражения WITH, которое может создать одно или несколько общих табличных выражений. Результат вычисления этого выражения можно рассматривать в качестве временной таблицы, существующей только во время выполнения всего запроса.

Основные преимущества использования CTE:

- Улучшение читаемости сложных запросов
- Возможность рекурсивных запросов
- Многократное использование в пределах одного запроса
- Помощь в организации сложной логики

CTE имеет следующий синтаксис:

```
WITH имя_cte (список_столбцов)
AS
(
    запрос_cte
)
основной_запрос;
```

3.2 Область видимости CTE внутри SQL-запроса

Область видимости Common Table Expressions строго ограничена контекстом одного SQL-запроса, в котором они определены. CTE существуют только в рамках выполнения того запроса, который непосредственно следует за блоком WITH, и не доступны за его пределами. CTE автоматически удаляются после его завершения

CTE видны только в основном запросе, следующем после блока WITH, и не доступны в других запросах той же сессии.

Внутри блока WITH можно определять несколько CTE, причем каждый последующий CTE может ссылаться на предыдущие, но не на последующие.

В рекурсивных CTE рекурсивная часть может ссылаться на собственное имя CTE, что создает специальную область видимости для итеративных вычислений.

Следует отметить, что CTE не сохраняют свое состояние между вызовами, даже если выполняется несколько одинаковых запросов подряд.

Эта ограниченная область видимости делает CTE идеальным инструментом для временной организации сложных запросов без загрязнения общего пространства базы данных.

3.3 Основные типы CTE

Основная функция общих табличных выражений – упрощать реализацию сложных SQL-запросов, разбивая их на составные части.

Рассмотрим пример использования CTE для структуризации запроса.

3.3.1 Использование CTE для структуризации запроса

Пример 1. Статистика успеваемости: средние баллы по студентам

Найти студентов, сдавших экзамены лучше среднего по группе.

```
-- CTE для вычисления средней оценки по каждой группе
WITH GroupAvg AS (
    SELECT studgroup, AVG(mark) as avg_grade
    FROM STUD JOIN EXAM ON STUD.studnum = EXAM.studnum
    GROUP BY studgroup
)
-- основной запрос, который выводит всех студентов,
-- получивших оценку выше, чем средняя по группе
SELECT studname, STUD.studgroup, EXAM.mark, avg_grade
FROM STUD JOIN EXAM ON STUD.studnum = EXAM.studnum
    JOIN GroupAvg ON STUD.studgroup = GroupAvg.studgroup
WHERE EXAM.mark > GroupAvg.avg_grade;
```

Поясним решение.

Данный SQL-запрос использует Common Table Expression для выявления студентов, показавших результаты выше среднего по своей учебной группе.

Сначала создается временный результат GroupAvg, который вычисляет среднюю оценку для каждой группы путем объединения таблиц студентов и экзаменов с последующей агрегацией данных.

Затем основной запрос отбирает конкретных студентов, соединяя исходные таблицы с рассчитанными средними значениями по группам, и применяет фильтрацию для вывода только тех записей, где индивидуальная оценка студента превышает средний показатель по его группе, что позволяет быстро идентифицировать наиболее успевающих учащихся в каждом учебном коллективе.

Такой подход не только улучшает читаемость запроса за счет разделения логики на понятные этапы, но и демонстрирует эффективное использование CTE для решения типовых аналитических задач в образовательной сфере.

3.3.2 Множественное использование CTE

Следующий пример демонстрирует применение множественных Common Table Expressions для комплексного анализа академической успеваемости.

Пример 2. Анализ успеваемости по предметам в разрезе групп

Получить комплексную картину академической успеваемости студентов по учебным группам и предметам для принятия управленческих решений.

```
-- Рассчитать базовую статистику по учебным группам
WITH GroupStats AS (
    SELECT studgroup, COUNT(*) as total_students
    FROM STUD
    GROUP BY studgroup),
-- Анализ успеваемости по предметам в разрезе групп
ExamStats AS (
    SELECT studgroup, subname,
           AVG(mark) as avg_mark, COUNT(*) as exams_count
    FROM STUD JOIN EXAM ON STUD.studnum = EXAM.studnum
           JOIN SUB ON SUB.subnum = EXAM.subnum
    GROUP BY studgroup, subname
)
-- Объединение статистик и формирование итогового отчета
SELECT GroupStats.studgroup, total_students, subname,
       avg_mark, exams_count
FROM ExamStats JOIN GroupStats
    ON ExamStats.studgroup = GroupStats.studgroup
ORDER BY ExamStats.studgroup, ExamStats.avg_mark DESC;
```

Поясним решение. На первом этапе формируется базовая статистика по учебным группам через CTE `GroupStats`, где подсчитывается общее количество студентов в каждой группе с помощью агрегирующей функции `COUNT`. Параллельно создается второй временный результат `ExamStats`, который анализирует успеваемость по предметам в разрезе групп, вычисляя средний балл и количество проведенных экзаменов путем соединения таблиц `STUD`, `EXAM` и `SUB` с последующей группировкой по учебным группам и названиям предметов.

На заключительном этапе происходит интеграция полученных статистик - основной запрос объединяет результаты обоих CTE через оператор `JOIN`, сопоставляя данные по полю `studgroup`, что позволяет сформировать сводный отчет, содержащий информацию о группе, общем количестве студентов, предмете, среднем балле и количестве студентов, сдавших экзамены по предмету.

Финальная сортировка по учебной группе и среднему баллу в порядке убывания обеспечивает удобное представление данных, где в рамках каждой группы предметы ранжируются по уровню успеваемости, что делает этот запрос ценным инструментом для методистов и администрации учебного заведения.

3.3.3 Рекурсивные запросы с использованием CTE

Общие табличные выражения используют для построения рекурсивных запросов. Использование в выражении WITH ключевого слова `RECURSIVE` в контексте CTE представляет собой механизм выполнения рекурсивных запросов, позволяющий обрабатывать иерархические и древовидные структуры данных средствами SQL. Это расширение стандарта SQL, реализованное в PostgreSQL и других современных СУБД.

Рекурсивный CTE состоит из двух обязательных частей:

- базовый случай - нерекурсивная часть, инициализирующая результат;
- рекурсивный случай - часть, ссылающаяся на собственное определение CTE.

Принцип выполнения поясняет следующий простейший синтаксис рекурсивного CTE.

```
WITH RECURSIVE имя_CTE
AS (
    -- Базовый случай (выполняется один раз)
    SELECT ... FROM ...
    UNION [ALL]
    -- Рекурсивный случай (выполняется итеративно)
    SELECT ... FROM имя_CTE JOIN ...
    -- основной запрос
    SELECT ... FROM имя_CTE;
```

Для соблюдения корректности рекурсивных CTE необходимо должны выполняться следующие требования:

- наличие начального набора данных;
- наличие ссылки на собственное имя CTE в рекурсивной части;
- наличие предиката, останавливающий рекурсию;
- каждая итерация должна приближать к условию остановки.

Для реализации следующего примера необходимо добавить в базу данных новую таблицу «Учебный план», к которой и построим рекурсивный запрос. Таблица содержит посеместровое распределение дисциплин с указанием признака обязательности изучения дисциплины или изучения ее по выбору студентов.

```
-- Таблица для хранения семестров и предметов в них
CREATE TABLE CURRICULUM (
    semester INTEGER NOT NULL,
    subnum INTEGER REFERENCES SUB(subnum),
    is_required BOOLEAN DEFAULT TRUE,
    PRIMARY KEY (semester, subnum)
);

-- Наполняем тестовыми данными таблицу CURRICULUM
INSERT INTO CURRICULUM (semester, subnum, is_required)
VALUES
    (1, 1, TRUE), -- 1 семестр: Математика (обязательная)
    (1, 2, TRUE), -- 1 семестр: Физика (обязательная)
    (2, 3, TRUE), -- 2 семестр: Программирование (обязательная)
    (2, 4, FALSE), -- 2 семестр: Базы данных (по выбору)
```

```
(3, 1, FALSE), -- 3 семестр: Математика (углубленный курс)
(4, 2, FALSE); -- 4 семестр: Физика (спецкурс)
```

Пусть содержимое таблицы SUB следующее:

	subnum [PK] integer	subname character (50)	subvolume integer
1	1	Математика ...	100
2	2	Физика ...	200
3	3	Программирование ...	200
4	4	Базы данных ...	1000

Пример 3. Анализ учебного плана по семестрам

Построить последовательное представление учебного процесса, отображающее все предметы, изучаемые в каждом семестре, с классификацией их типа (обязательные/по выбору) и определением последовательности изучения.

Проанализируем задачу. Требуется начать анализ с первого семестра учебного плана и последовательно переходить от текущего семестра к следующему. При этом будем анализировать не более 6 семестров для предотвращения бесконечной рекурсии.

В качестве результата запрос должен выводить: номер семестра, название предмета, тип предмета (обязательный/по выбору) и позицию в последовательности.

```
WITH RECURSIVE StudyPath AS (
    -- Базовый случай: начинаем с первого семестра
    SELECT semester, SUB.subnum, subname, is_required,
           -- вводим счетчик шагов рекурсии
           1 as path_length
    FROM CURRICULUM JOIN SUB ON CURRICULUM.subnum=SUB.subnum
    -- Начинаем с первого семестра
    WHERE CURRICULUM.semester = 1
    UNION
    -- Рекурсивный случай: переходим к следующему семестру
    SELECT CURRICULUM.semester, SUB.subnum, SUB.subname,
           is_required, StudyPath.path_length + 1
    FROM CURRICULUM JOIN SUB ON CURRICULUM.subnum=SUB.subnum
           -- Следующий семестр
           JOIN StudyPath
           ON CURRICULUM.semester=StudyPath.semester+1
    -- Ограничиваем глубину (6 семестров)
    WHERE StudyPath.path_length < 6
)
SELECT semester as Семестр, subname as Предмет,
       CASE
           WHEN is_required THEN 'Обязательный'
           ELSE 'По выбору'
       END as "Тип предмета",
       path_length as "Длина пути"
FROM StudyPath
ORDER BY semester, is_required DESC;
```

Поясним выполнение рекурсивного запроса. Данный рекурсивный запрос последовательно строит учебный план по семестрам, начиная с первого семестра. На каждом шаге рекурсии происходит переход к следующему семестру, где добавляются соответствующие учебные предметы с указанием их типа - обязательные или по выбору. Счетчик `path_length` отслеживает глубину рекурсии, соответствуя номеру текущего семестра, что позволяет ограничить анализ шестью семестрами для предотвращения бесконечной рекурсии. Использование `UNION` вместо `UNION ALL` гарантирует устранение дубликатов, обеспечивая уникальность каждой комбинации семестр-предмет в финальном результате, который выводится в удобном для анализа формате с сортировкой по семестрам и типам предметов.

	Семестр integer	Предмет character (50)	Тип предмета text	Длина пути integer
1	1	Математика ...	Обязательный	1
2	1	Физика ...	Обязательный	1
3	2	Программиро...	Обязательный	2
4	2	Базы данных ...	По выбору	2
5	3	Математика ...	По выбору	3
6	4	Физика ...	По выбору	4

Пример 4. Анализ распределения учебной нагрузки по семестрам

Разработать аналитический отчет, отображающий структуру учебного плана по семестрам с количественной оценкой учебной нагрузки и качественной характеристикой интенсивности обучения.

Для решения данной задачи необходимо сгенерировать последовательность семестров от первого до пятого, рассчитать количество учебных дисциплин в каждом из них и предоставить качественную характеристику интенсивности обучения, классифицируя нагрузку как отсутствующую (нет предметов), минимальную (один предмет), умеренную (два предмета) или высокую (более двух предметов). Полученные результаты должны быть визуализированы в структурированном отчете, позволяющем академическому руководству принимать обоснованные решения по оптимизации учебных планов и перераспределению академической нагрузки между семестрами.

```
WITH RECURSIVE SemesterSequence AS (
  -- Базовый случай: первый семестр
  SELECT 1 as semester,
    (SELECT COUNT(*)
     FROM CURRICULUM
     WHERE semester = 1) as subject_count
  UNION ALL
  -- Рекурсивный случай: следующий семестр
  SELECT SemesterSequence.semester + 1,
    (SELECT COUNT(*)
     FROM CURRICULUM
     WHERE semester = SemesterSequence.semester + 1)
  FROM SemesterSequence
  -- Показываем до 5 семестра
  WHERE SemesterSequence.semester < 5)
```

```

SELECT
    semester as Семестр,
    subject_count as "Количество предметов",
    CASE
        WHEN subject_count = 0 THEN 'Нет предметов'
        WHEN subject_count = 1 THEN 'Один предмет'
        WHEN subject_count = 2 THEN 'Два предмета'
        ELSE 'Более двух предметов'
    END as Описание
FROM SemesterSequence;

```

Результат выполнения запроса при заданном состоянии базы данных следующий:

	Семестр integer	Количество предметов bigint	Описание text
1	1	2	Два предмета
2	2	2	Два предмета
3	3	1	Один предмет
4	4	1	Один предмет
5	5	0	Нет предметов

3.3.4 CTE с модификацией данных

CTE в PostgreSQL представляют собой мощный инструмент не только для сложных запросов и рекурсивных операций, но и для организации операций модификации данных. Использование CTE при выполнении INSERT, UPDATE и DELETE операций позволяет создавать более структурированные, читаемые и безопасные запросы, разделяя сложную логику на понятные этапы.

Особую ценность CTE приобретают в сценариях, требующих последовательного выполнения нескольких модифицирующих операций, где результат одного этапа становится основой для следующего. Это обеспечивает атомарность выполнения и целостность данных, что критически важно в производственных средах.

В следующем примере рассматривается практическое применение CTE для операции удаления устаревших данных с обязательным предварительным сохранением информации в архиве. Такой подход демонстрирует, как CTE могут использоваться для реализации сложной бизнес-логики, включающей несколько этапов модификации данных, при сохранении прозрачности и контроле над процессом.

Этот пример особенно актуален для задач регулярного обслуживания баз данных, обеспечения соответствия политикам хранения информации и оптимизации производительности систем за счет управления объемом хранимых данных.

Пример 5. Архивация данных успеваемости

Реализовать процедуру безопасного удаления устаревших записей об экзаменах проведенных до 2025 года, с обязательным предварительным сохранением данных в архивной таблице.

Для реализации следующего примера необходимо добавить в базу данных новую таблицу для архивных записей об экзаменах.

```
-- Создаем таблицу для архивных записей об экзаменах
CREATE TABLE EXAM_ARCHIVE AS TABLE EXAM WITH NO DATA;
```

Для решения задачи необходимо идентифицировать все экзаменационные записи с неудовлетворительными оценками, соответствующие заданному временному периоду, и обеспечить их безопасное перемещение в архив перед окончательным удалением из основной таблицы. Процедура должна гарантировать целостность данных и предоставлять детальный отчет об удаленных записях для подтверждения корректности выполнения операции.

```
WITH ExamsToDelete AS (
    SELECT studnum, subnum, examdate, mark
    FROM EXAM
    WHERE examdate < '2025-01-01' AND mark = 2),
ArchiveBeforeDelete AS (
    INSERT INTO EXAM_ARCHIVE (studnum, subnum, examdate, mark)
    SELECT studnum, subnum, examdate, mark
    FROM EXAMSTODELETE
    RETURNING studnum, subnum, examdate )
DELETE
FROM exam
WHERE EXISTS (SELECT 1
               FROM EXAMSTODELETE
               WHERE exam.studnum = ExamsToDelete.studnum
                  AND exam.subnum = ExamsToDelete.subnum
                  AND exam.examdate = ExamsToDelete.examdate)
RETURNING studnum, subnum, examdate, mark;
```

В данном примере оператор RETURNING используется для возврата информации о строках, которые были затронуты операциями вставки и удаления.

В таблицу EXAM_ARCHIVE будут помещены все удаляемые из таблицы EXAM строки,

	studnum integer	subnum integer	examdate date	mark integer
1	3	1	2024-06-10	2

а таблица EXAM примет следующий вид:

	studnum integer	subnum integer	examdate date	mark integer
1	1	1	2024-06-10	5
2	1	2	2024-06-20	4
3	1	3	2024-06-30	3
4	2	1	2024-06-10	5
5	2	2	2024-06-20	5

4 Практическая часть

Разработайте 4 типа запроса с использованием общих табличных выражений СТЕ в соответствии с вариантом задания:

1. Простой СТЕ для анализ превышения среднего показателя.
2. Множественные СТЕ для комплексного анализа по двум измерениям
3. Рекурсивный СТЕ для формироавния отчета с накоплением за период
4. СТЕ с модификацией данных для архивация устаревших записей

Вариант 1: Склад

1. Найти товары, количество которых на складе превышает среднее количество по всем товарам.
2. Проанализировать распределение товаров по складам и группам, вывести общую статистику по наличию.
3. Построить отчет о движении товаров за последние три дня с накоплением остатков.
4. Реализовать архивацию товаров, которые отсутствуют на складах более 30 дней.

Вариант 2: Производство продуктов питания

1. Найти продукты, объем производства которых выше среднего по всем продуктам.
2. Проанализировать объемы производства по фирмам и группам продуктов.
3. Построить отчет о динамике производства за последние три года с кумулятивными итогами.
4. Архивировать записи о производстве продуктов, которые не производились более двух лет.

Вариант 3: Учебная нагрузка

1. Найти преподавателей, нагрузка которых превышает среднюю по кафедре.
2. Проанализировать распределение нагрузки по кафедрам и направлениям дисциплин.
3. Построить последовательность семестров с количеством часов нагрузки по преподавателям.
4. Архивировать записи о нагрузке за семестры, которые завершились более трех лет назад.

Вариант 4: Растениеводство

1. Найти культуры, урожайность которых выше средней по всем культурам.
2. Проанализировать урожайность по районам и семействам культур.
3. Построить отчет об изменении урожайности за последние три года с накоплением.
4. Архивировать данные об урожайности за годы, предшествующие последним трем.

Вариант 5: Этнический состав стран

1. Найти национальности, численность которых в странах превышает среднюю.
2. Проанализировать этнический состав по странам и расам.
3. Построить отчет об изменении этнического состава за последние три года.
4. Архивировать данные об этническом составе за годы, предшествующие последним трем.

Вариант 6: Курсы иностранных языков

1. Найти учащихся, которые посещают курсы дольше среднего срока.
2. Проанализировать популярность языков и языковых групп среди учащихся.
3. Построить отчет о посещаемости занятий по месяцам с накоплением.
4. Архивировать записи о посещении занятий, которые завершились более года назад.

Вариант 7: Услуги парикмахера

1. Найти клиентов, потративших на услуги больше средней суммы.
2. Проанализировать распределение услуг по типам и клиентам.
3. Построить отчет о доходах от услуг по дням с кумулятивными итогами.
4. Архивировать записи о предварительной записи клиентов старше шести месяцев.

Вариант 8: Ателье по пошиву и ремонту одежды

1. Найти виды услуг, стоимость которых превышает среднюю по ателье.
2. Проанализировать объемы услуг по ателье и типам услуг.
3. Построить отчет о стоимости услуг за последние три месяца с накоплением.
4. Архивировать записи об услугах, оказанных более года назад.

Вариант 9: Красная книга растений

1. Найти растения, количество которых в странах превышает среднее.
2. Проанализировать распространение растений по странам и семействам.
3. Построить отчет об изменении численности растений за последние три года.
4. Архивировать данные о произрастании растений за годы, предшествующие последним трем.

Вариант 10: Животноводство

1. Найти виды скота, поголовье которых в совхозах превышает среднее.
2. Проанализировать поголовье скота по совхозам и породам.
3. Построить отчет об изменении поголовья за последние три года с накоплением.
4. Архивировать записи о поголовье скота за годы, предшествующие последним трем.

Вариант 11: Языки народов мира

1. Найти языки, численность носителей которых в странах превышает среднюю по всем языкам.
2. Проанализировать распространение языков по странам и языковым группам.
3. Построить отчет об изменении численности носителей языков за последние три года с накоплением.
4. Архивировать записи о численности носителей языков за годы, предшествующие последним трем.

Вариант 12: Сеть компьютерных магазинов города

1. Найти комплектующие, количество которых в магазинах превышает среднее по всем комплектующим.
2. Проанализировать наличие комплектующих по магазинам и видам.
3. Построить отчет об изменении наличия комплектующих за последние три года с накоплением.
4. Архивировать записи о наличии комплектующих за годы, предшествующие последним трем.

Вариант 13: Туристические фирмы города

1. Найти клиентов, потративших на туры больше средней суммы по всем клиентам.
2. Проанализировать продажи туров по фирмам и странам назначения.
3. Построить отчет об изменении стоимости проданных туров за последние три года с накоплением.
4. Архивировать записи о турах, которые состоялись более трех лет назад.

Вариант 14: Подписка на периодические издания организациями города

1. Найти издания, на которые подписано больше организаций, чем в среднем по всем изданиям.
2. Проанализировать подписку по организациям и типам изданий.
3. Построить отчет об изменении количества подписок за последние три года с накоплением.
4. Архивировать записи о подписках, которые закончились более трех лет назад.

Вариант 15: Расписание занятий в вузе

1. Найти преподавателей, у которых количество часов превышает среднее по кафедре.
2. Проанализировать распределение часов по дисциплинам и дням недели.
3. Построить отчет о нагрузке преподавателей по семестрам с накоплением часов.
4. Архивировать записи о занятиях, которые прошли более трех лет назад.

Вариант 16: Справочная служба аптек

1. Найти лекарства, количество упаковок которых в аптеках превышает среднее по всем лекарствам.
2. Проанализировать наличие лекарств по аптекам и группам.
3. Построить отчет об изменении наличия лекарств за последние три дня с накоплением.
4. Архивировать записи о наличии лекарств за дни, предшествующие последним трем.

Вариант 17: Сеть магазинов по продаже бытовой техники

1. Найти технику, количество которой в магазинах превышает среднее по всей технике.
2. Проанализировать наличие техники по магазинам и маркам.
3. Построить отчет об изменении наличия техники за последние три дня с накоплением.
4. Архивировать записи о наличии техники за дни, предшествующие последним трем.

Вариант 18: Стоматологическая поликлиника

1. Найти пациентов, потративших на услуги больше средней суммы по всем пациентам.
2. Проанализировать оказание услуг по видам и пациентам.
3. Построить отчет об изменении доходов от услуг за последние три года с накоплением.
4. Архивировать записи об услугах, оказанных более трех лет назад.

Вариант 19: Строительство жилья

1. Найти подрядчиков, построивших больше квартир, чем в среднем по всем подрядчикам.
2. Проанализировать строительство по подрядчикам и заказчикам.
3. Построить отчет об изменении количества сданных квартир за последние три года с накоплением.
4. Архивировать записи о зданиях, сданных более трех лет назад.

Вариант 20: Сотовые и интернет компании России

1. Найти компании, количество абонентов которых в городах превышает среднее по всем компаниям.
2. Проанализировать количество абонентов по компаниям и городам.
3. Построить отчет об изменении количества абонентов за последние три года с накоплением.
4. Архивировать записи о количестве абонентов за годы, предшествующие последним трем.

Вариант 21: Поставка товаров

1. Найти товары, объем поставок которых превышает средний по всем товарам.
2. Проанализировать объемы поставок по поставщикам и товарам.
3. Построить отчет об изменении объемов поставок за последний год с накоплением.
4. Архивировать записи о поставках, которые состоялись более трех лет назад.

Вариант 22: Киносеанс

1. Найти кинофильмы, количество сеансов которых превышает среднее по всем фильмам.
2. Проанализировать расписание сеансов по кинотеатрам и фильмам.
3. Построить отчет об изменении доходов от сеансов за последний месяц с накоплением.
4. Архивировать записи о сеансах, которые прошли более года назад.

Вариант 23: Поликлиника

1. Найти врачей, количество приемов у которых превышает среднее по всем врачам.
2. Проанализировать количество приемов по специализациям врачей и пациентам.
3. Построить отчет об изменении страховых выплат за последний год с накоплением.
4. Архивировать записи о приемах, которые состоялись более трех лет назад.

Вариант 24: Автосервис

1. Найти мастеров, стоимость выполненных ремонтов у которых превышает среднюю по всем мастерам.
2. Проанализировать объемы ремонтов по специализациям мастеров и маркам автомобилей.
3. Построить отчет об изменении доходов от ремонтов за последний год с накоплением.
4. Архивировать записи о ремонтах, которые были завершены более двух лет назад.

Вариант 25: Предприятие

1. Найти сотрудников, оплата за проекты у которых превышает среднюю по всем сотрудникам.
2. Проанализировать распределение проектов по сотрудникам и их длительность.
3. Построить отчет об изменении стоимости проектов за последние три года с накоплением.
4. Архивировать записи о проектах, которые были завершены более пяти лет назад.

Вариант 26: Библиотека

1. Найти книги, количество выдач которых превышает среднее по всем книгам.
2. Проанализировать выдачу книг по абонентам и авторам.
3. Построить отчет об изменении количества выдач за последний год с накоплением.
4. Архивировать записи о выдачах книг, которые были возвращены более трех лет назад.

Вариант 27: Сбор лекарственных трав аптекой

1. Найти травы, объем сбора которых превышает средний по всем травам.
2. Проанализировать объемы сбора по сборщикам и видам трав.
3. Построить отчет об изменении объемов сбора за последний год с накоплением.
4. Архивировать записи о сдаче трав, которые состоялись более двух лет назад.

Вариант 28: Увлечения

1. Найти клубы, количество участников в которых превышает среднее по всем клубам.
2. Проанализировать участие студентов в клубах по направлениям деятельности.
3. Построить отчет об изменении вступительных взносов за последний год с накоплением.
4. Архивировать записи о вступлении в клубы, которые состоялись более трех лет назад.

Вариант 29: Красная книга животных

1. Найти животных, популяция которых в странах превышает среднюю по всем животным.
2. Проанализировать популяцию животных по странам и классам.
3. Построить отчет об изменении популяции за последние пять лет с накоплением.
4. Архивировать записи о популяции животных за годы, предшествующие последним пяти.

Вариант 30: Мировые ресурсы

1. Найти ресурсы, объем добычи которых в странах превышает средний по всем ресурсам.
2. Проанализировать добычу ресурсов по странам и видам.
3. Построить отчет об изменении объемов добычи за последние три года с накоплением.
4. Архивировать записи о добыче ресурсов за годы, предшествующие последним трем.

5. Контрольные вопросы

1. Что такое Common Table Expression и каков базовый синтаксис его создания в PostgreSQL?
2. В чем основное отличие обычного CTE от рекурсивного CTE?
3. Какие ключевые преимущества дает использование CTE по сравнению с вложенными подзапросами?
4. Какова область видимости CTE внутри SQL-запроса?
5. Каков обязательный синтаксис рекурсивного CTE и из каких двух частей он состоит?
6. Как работает механизм выполнения рекурсивного CTE на каждом шаге итерации?
7. Какие обязательные условия должны быть выполнены для корректной работы рекурсивного CTE?
8. Как предотвратить бесконечную рекурсию в рекурсивных запросах?
9. В каких практических сценариях наиболее целесообразно применение обычных CTE?
10. В каких практических сценариях наиболее целесообразно применение рекурсивных CTE?
11. Как использовать CTE в операциях модификации данных (INSERT, UPDATE, DELETE)?
12. Как использовать несколько CTE в одном запросе, и каков порядок их обработки?